

BCSE305L-EMBEDDED SYSTEM DESIGN

Embedded Systems

MODULE-1

Embedded System Product Development

- Characteristics of embedded systems, general purpose, customized, application specific processors, Embedded product development cycle.

EMBEDDED SYSTEM

WHAT IS EMBEDDED SYSTEM?

Embedded system = Hardware + Software

Designed for specific purpose

An embedded system is a combination of computer hardware and software, designed for a specific function or functions within a larger system.

WHAT IS EMBEDDED SYSTEM?

S/W

```
#include "mbed.h"
#include "C12832.h"
#include "Sht31.h"

C12832 lcd(SPI_MOSI, SPI_SCK, SPI_MISO, p8, p11);
Sht31 sht31(I2C_SDA, I2C_SCL);
DigitalOut led(LED1);

int main() {
    printf("Set the temperature above 25 degrees to trigger the warning LED\n");

    while (1) {
        lcd.cls();

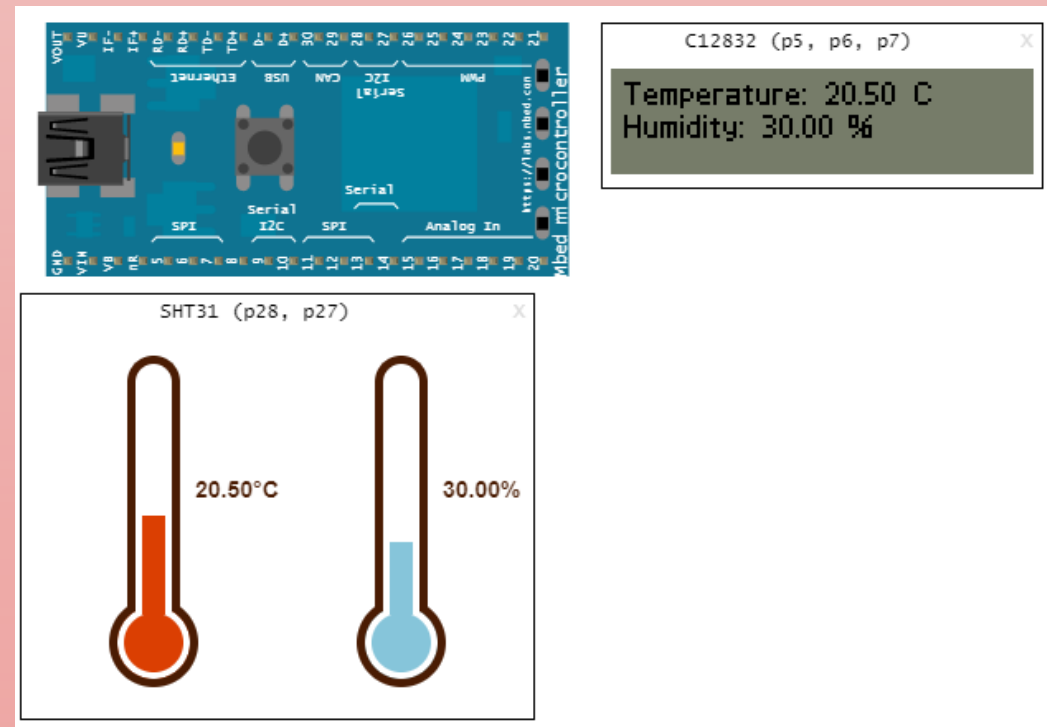
        float temp = sht31.readTemperature();
        float humidity = sht31.readHumidity();

        lcd.locate(3, 3);
        lcd.printf("Temperature: %.2f C", temp);
        lcd.locate(3, 13);
        lcd.printf("Humidity: %.2f %%", humidity);

        // turn on LED if the temperature is above 25 degrees
        led = temp > 25.0f;

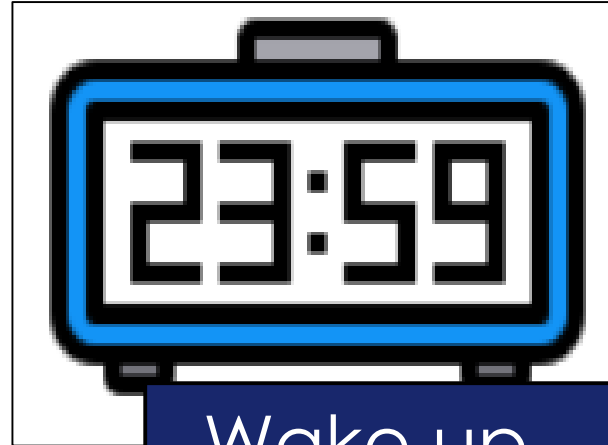
        wait(0.5f);
    }
}
```

H/W



Embedded System

Embedded system in daily life....



Wake up...



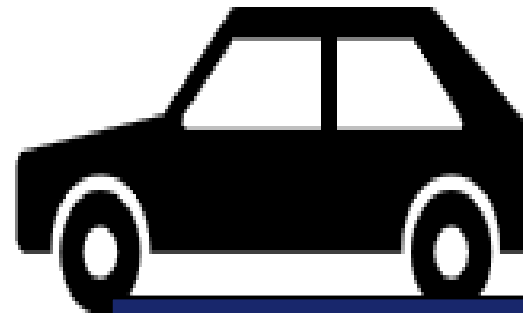
Washing...



Breakfast...



Elevator..



Car...

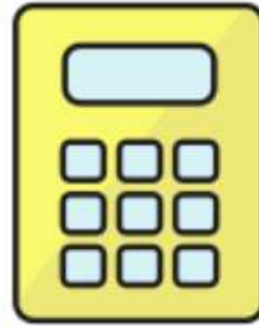


Traffic lights...

Inside a classroom...



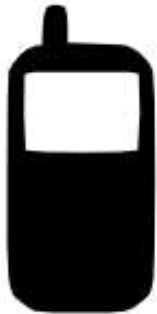
Smart watch...



Calculator...



Projector...



Mobile phone...



Smart shoe...



Wireless mic...

Back home.....



Smart TV...



Set top box...



Headphones...



Digital Camera...

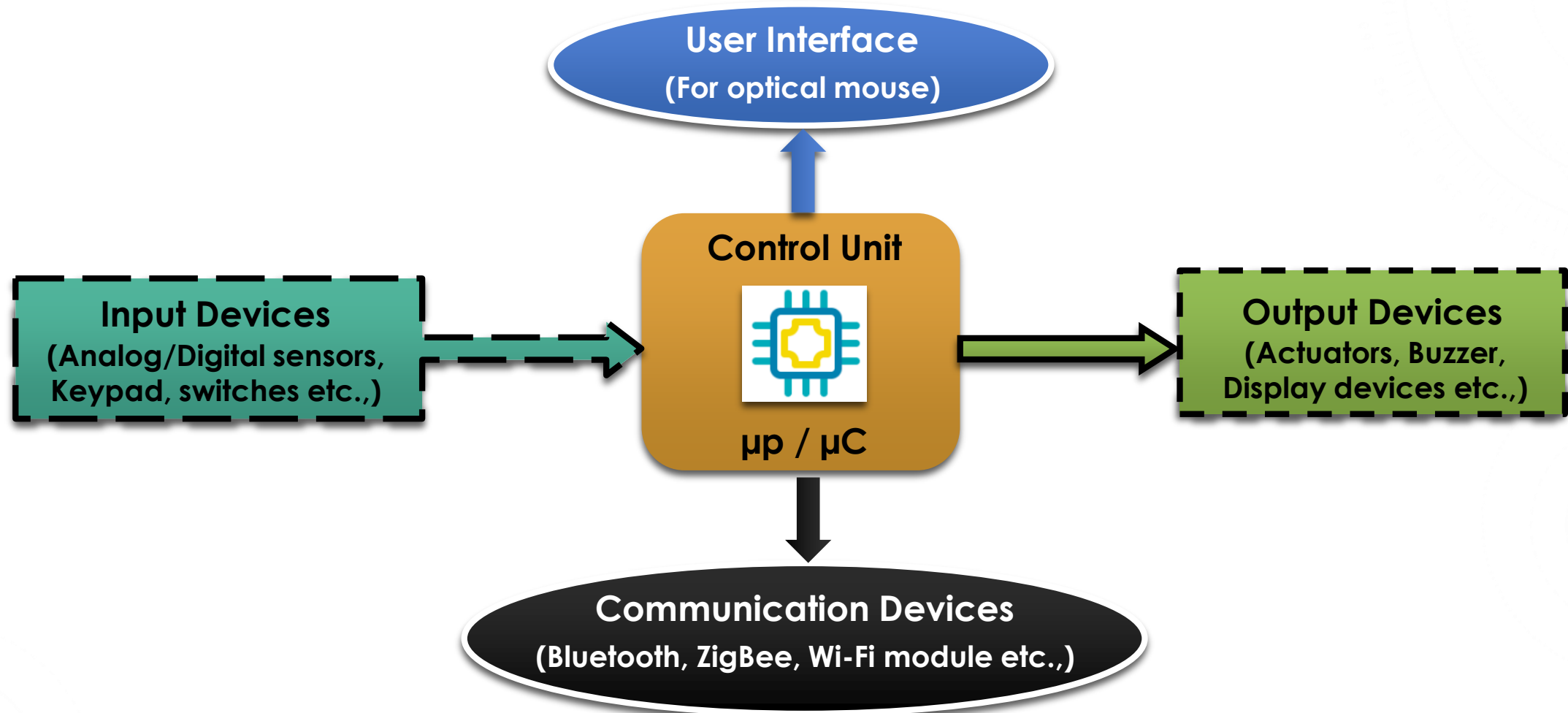


Printer...



CCTV...

EMBEDDED SYSTEM – BLOCK DIAGRAM



EMBEDDED SYSTEM - EXAMPLES

PURPOSE OF AN EMBEDDED SYSTEM

- ❑ **Each embedded system is designed to serve the purpose of any one or combinations of following task:**
 - **Data collection / storage / representation**
 - **Data communication**
 - **Data (signal) processing**
 - **Monitoring**
 - **Controlling**
 - **Application specific user interface**

EMBEDDED SYSTEM - EXAMPLES

MAJOR APPLICATION DOMAINS OF EMBEDDED SYSTEM

- ☐ Home Appliances
- ☐ Consumer Electronics
- ☐ Home Automation / Security
- ☐ Computer components
- ☐ Automotive
- ☐ Health care
- ☐ Military
- ☐ Industrial
- ☐ Business
- ☐ Retail
- ☐ Communications
- ☐ Avionics
- ☐ Banking

EMBEDDED SYSTEM - CHARACTERISTICS

EMBEDDED SYSTEM - CHARACTERISTICS

- ☐ Dedicated to specific tasks.
- ☐ Supported by a wide array of processors and processor architectures.
- ☐ Usually cost sensitive.
- ☐ Real-time constraints.
- ☐ Require specialized tools and methods.

EMBEDDED SYSTEM - CHARACTERISTICS

- ❑ Software failure is much more severe.
- ❑ Power & memory constraints.
- ❑ Operate under extreme environmental conditions.
- ❑ Fewer system resources.
- ❑ Real Time Operating System

WHY IS EMBEDDED SYSTEM DIFFERENT?

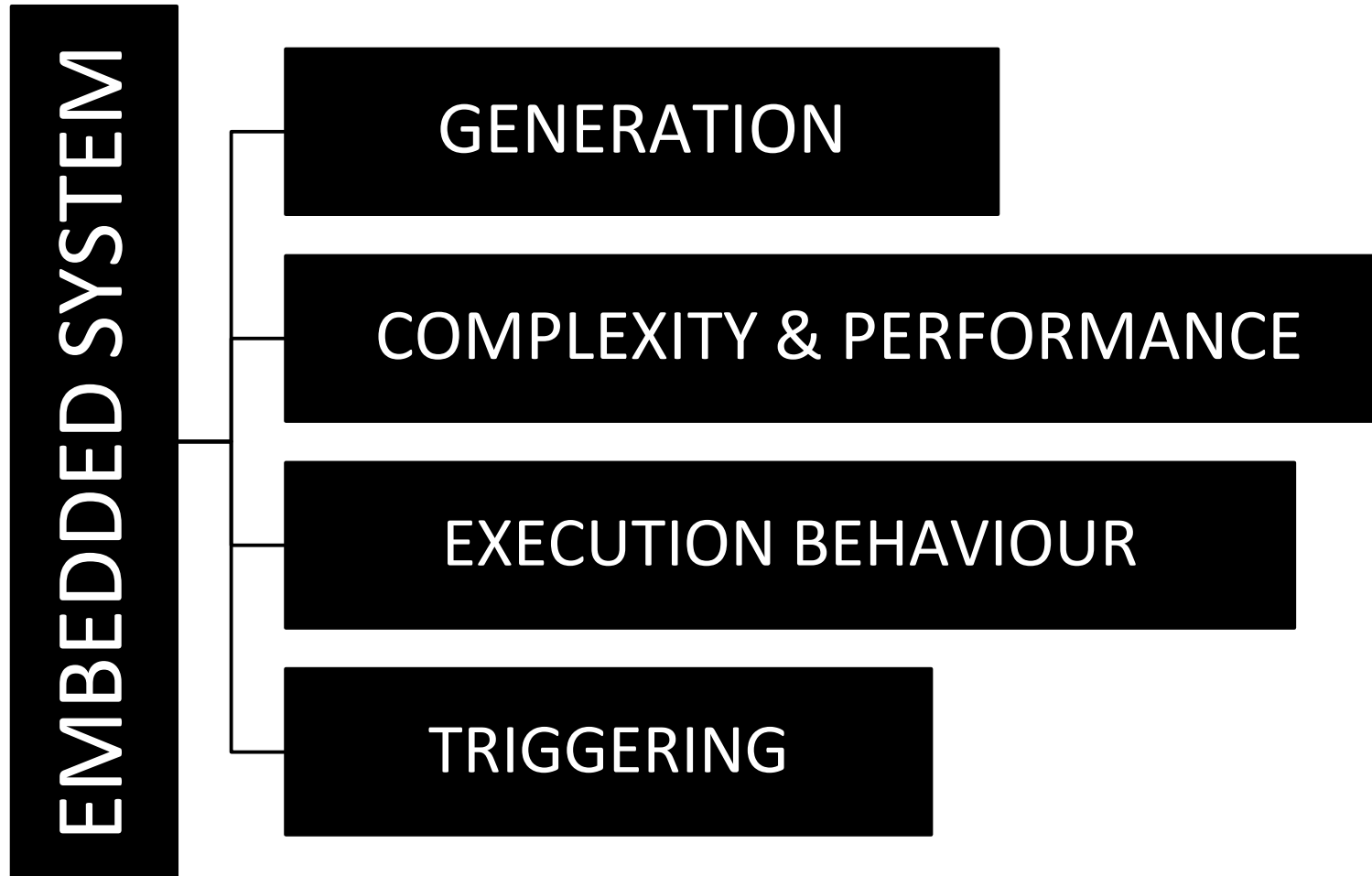
- ❑ **Complex algorithms – (Automotive Engine ECU)**
- ❑ **User interface – (GPS)**
- ❑ **Real time – (ABS)**
- ❑ **Multirate – (Multimedia Application)**
- ❑ **Manufacturing cost – (Mobile Phone)**
- ❑ **Power and energy – (Data Logger)**

GENERAL PURPOSE vs EMBEDDED SYSTEM

S. NO.	GENERAL PURPOSE COMPUTING SYSTEM	EMBEDDED SYSTEM
1	A system which is a combination of generic hardware and general purpose operating system for executing variety of applications	A system which is a combination of special purpose hardware and embedded OS for executing a specific set of application
2	Contains General Purpose Operating System (GPOS)	May or may not contain operating system for function
3	Applications can be modified by user	The firmware of the embedded system is pre-programmed and it is non-modifiable by the end user
4	Performance is a key deciding factor in the selection of the system. Always "Faster is better".	Application specific requirements (like performance, power, memory, cost) are key deciding factor
5	Need not to be deterministic in execution behavior	Execution behavior is deterministic for certain type of system like "hard real time systems" (Missile guidance or airbag system)
6	Personal computer, Laptops	DVD player, Digital camera

EMBEDDED SYSTEM - CLASSIFICATIONS

EMBEDDED SYSTEM - CLASSIFICATIONS



EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON GENERATION

❑ First Generation:

- build around 4-bit and 8-bit processor like 8085, Z80
- Simple hardware & firmware developed in assembly code
- Ex: Digital telephone keypads, calculator etc.,

❑ Second Generation:

- build around 8-bit and 16-bit processor
- complex and powerful instruction set
- Ex: Data acquisition system, SCADA system etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON GENERATION

❑ Third Generation:

- build around 16-bit and 32-bit processor
- application specific processor DSP & ASIC were introduced
- complex and powerful instruction set with pipelining
- dedicated embedded real-time OS entered into market
- Ex: robotics, industrial process control, networking etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON GENERATION

❑ Fourth Generation:

- advent of SOC, reconfigurable & multicore processors
- high performance, tight integration and miniaturization μ P
- High performance RTOS for functioning
- Ex: smart phone, PDA etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON COMPLEXITY AND SYSTEM PERFORMANCE

❑ Small-scale:

- simple in application needs
- performance requirements are not time critical
- build around low cost & low performance 8 or 16-bit $\mu p / \mu C$
- Ex: Electronic toys, digital multimeters etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON COMPLEXITY AND SYSTEM PERFORMANCE

❑ Medium-scale:

- slightly complex in hardware & software requirements
- build on medium performance & low cost 16/32-bit μ p / μ C
- usually contains embedded operating system
- Ex: DVD players, network routers etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON COMPLEXITY AND SYSTEM PERFORMANCE

❑ Large-scale:

- highly complex hardware and software requirement
- employed in mission critical application demanding high performance
- build around high performance 32 or 64-bit μp / μC or SOC or PLDs or multi-core processors
- may contain complex high performance RTOS
- Ex: Missile navigation system, MRI scanner, ABS etc.,

EMBEDDED SYSTEM - CLASSIFICATIONS

BASED ON EXECUTION BEHAVIOUR

☐ Deterministic:

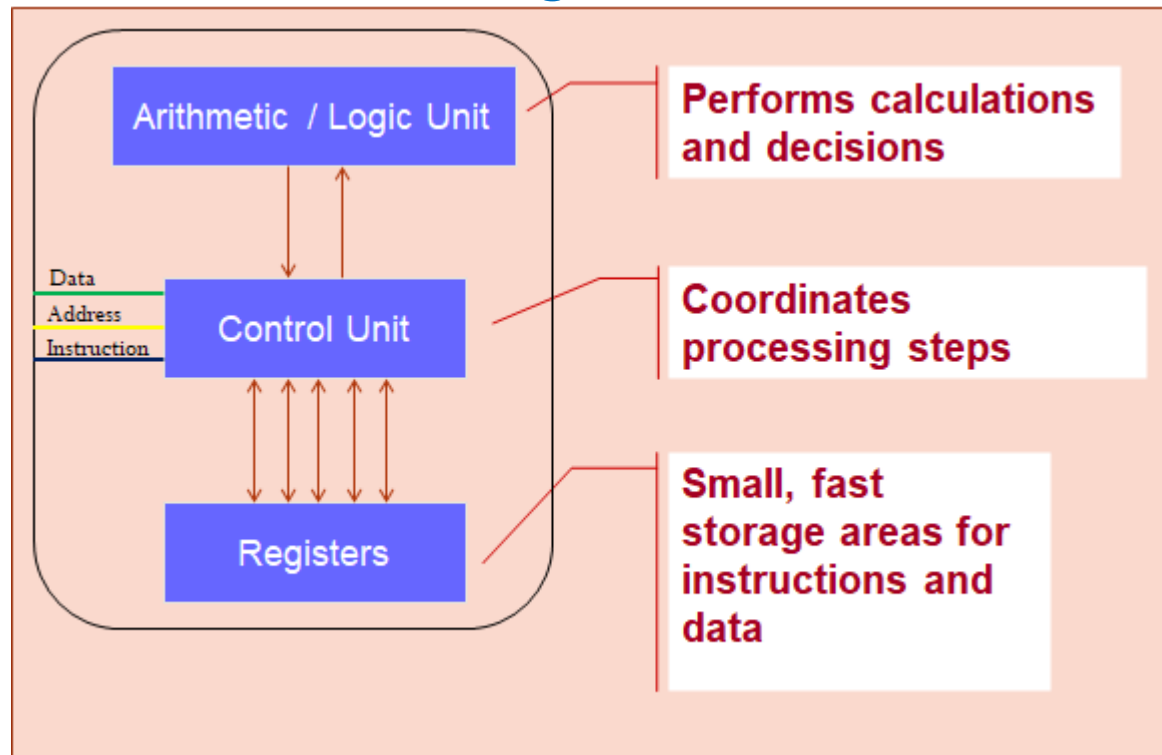
- Real-time systems
- Non Real-time systems

☐ Triggering mechanism:

- event triggered
- time triggered

WHAT IS MICROPROCESSOR?

It is a central processing unit (CPU) on a single integrated circuit (IC) chip containing millions of very small components including transistors, resistors, and diodes that work together.

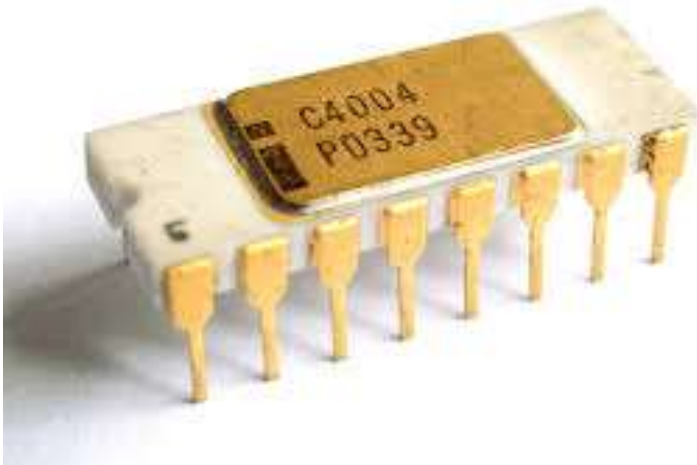


Examples: 8085, 8086, Intel i series, AMD, ARM, Qualcomm Snapdragon

WHAT IS THE FIRST MICROPROCESSOR?

History behind first microprocessor?

Intel 4004



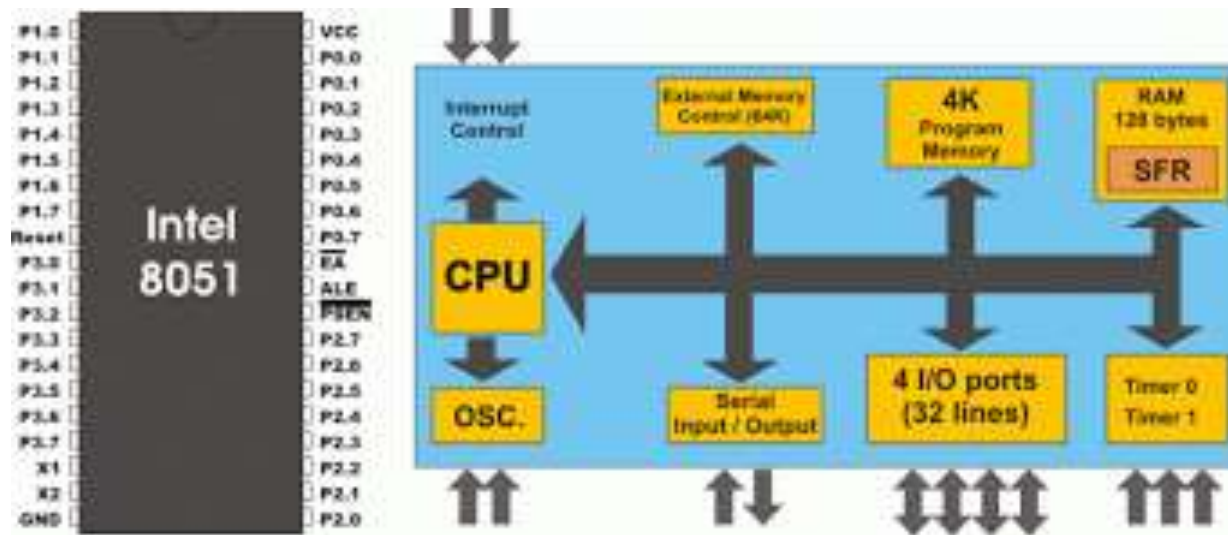
- **1969: The assignment** - Nippon Calculating Machine Corporation approached Intel to design 12 custom chips for its new Busicom 141-PF* printing calculator.
- **The Intel solution** - Intel designed a set of four chips known as the MCS-4. It included **CPU(4004)**, Shift registers(4003) RAM(4002), ROM(4001)
- **1971: Era of integrated electronics** - Intel purchased the rights from Nippon Calculating Machine Corporation and launched the Intel 4004 processor and its chipset with an advertisement in the November 15, 1971

APPLICATIONS OF MICROPROCESSOR

- ❖ Laptop
- ❖ Servers
- ❖ Networking devices
- ❖ Desktops
- ❖ Mobile phones
- ❖ Games machine
- ❖ Thin clients
- ❖ Military applications

WHAT IS MICROCONTROLLER?

- ❖ A microcontroller is a compact integrated circuit designed to govern a specific operation in an embedded system.
- ❖ A typical microcontroller includes a processor, memory and input/output (I/O) peripherals on a single chip.



Examples: Intel 8051, PIC 16F877, AVR32, ARM LPC2148

COMMONLY USED MICROCONTROLLER



Atmel AVR



AVR



ATX Mega



ATmega 328P



PIC 18F877A



8051

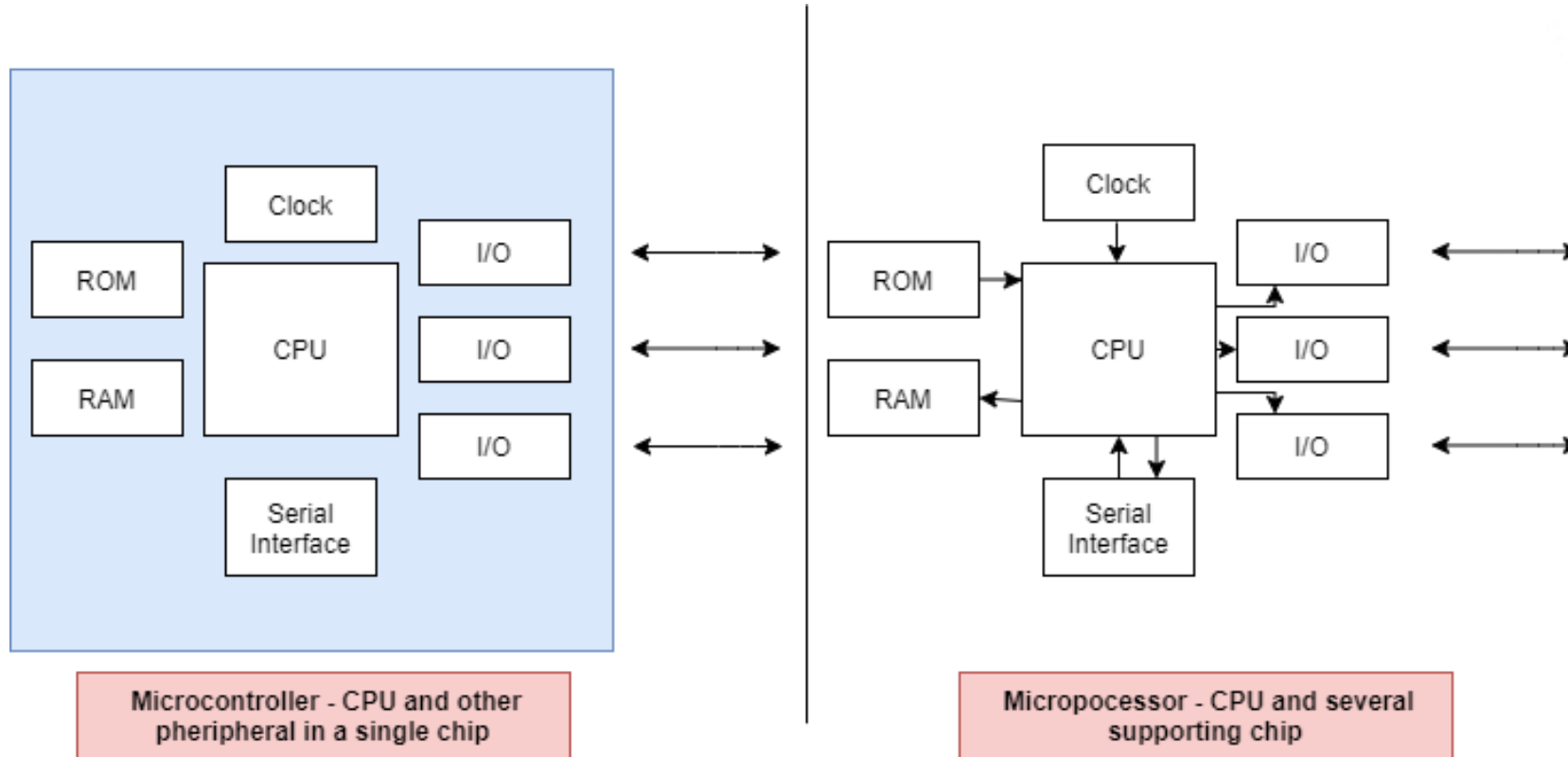


Arduino



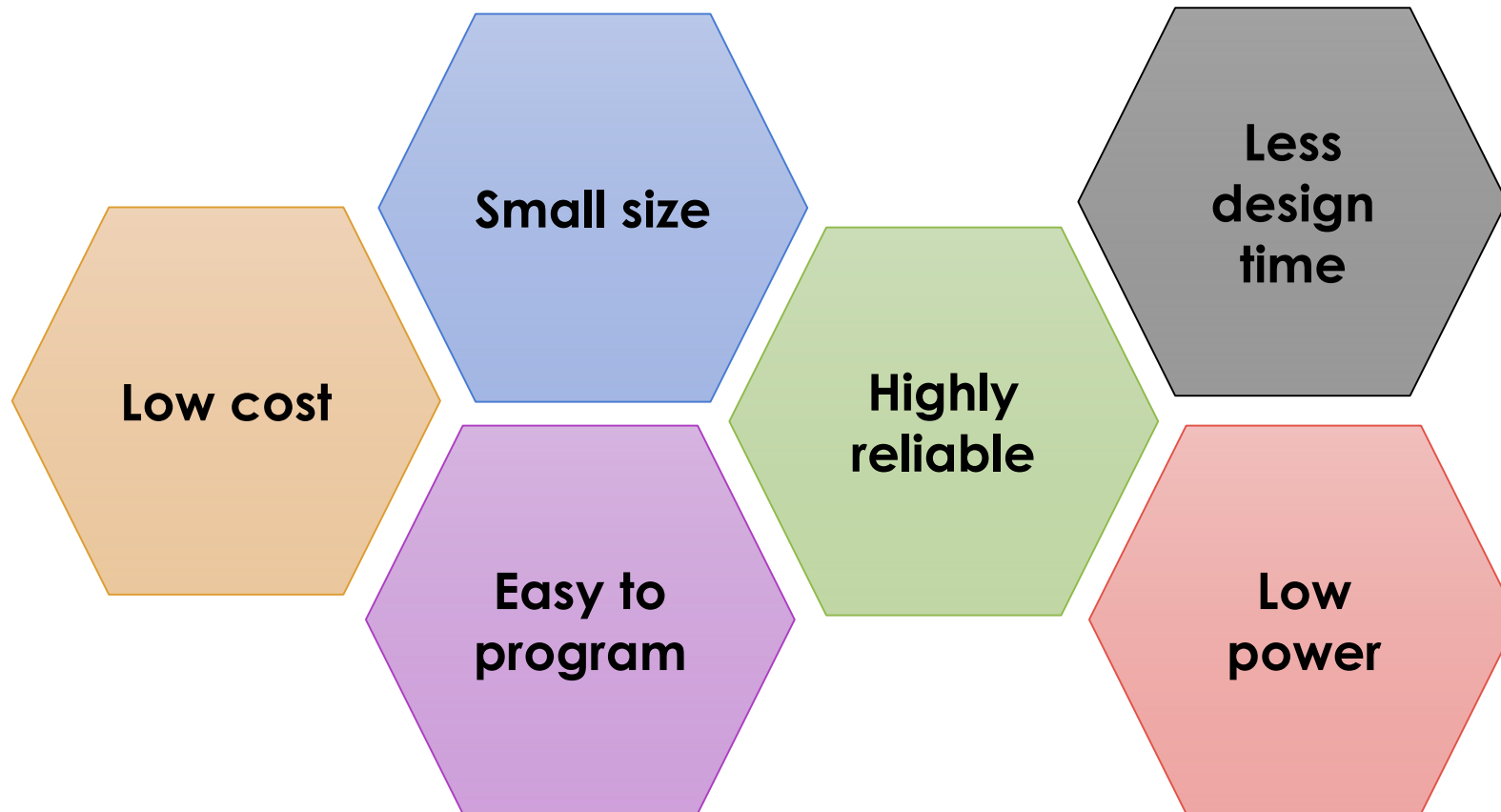
ARM

MP vs MC



WHY MICROCONTROLLERS IN ES?

Most of the embedded system uses Microcontrollers than Microprocessor !!!

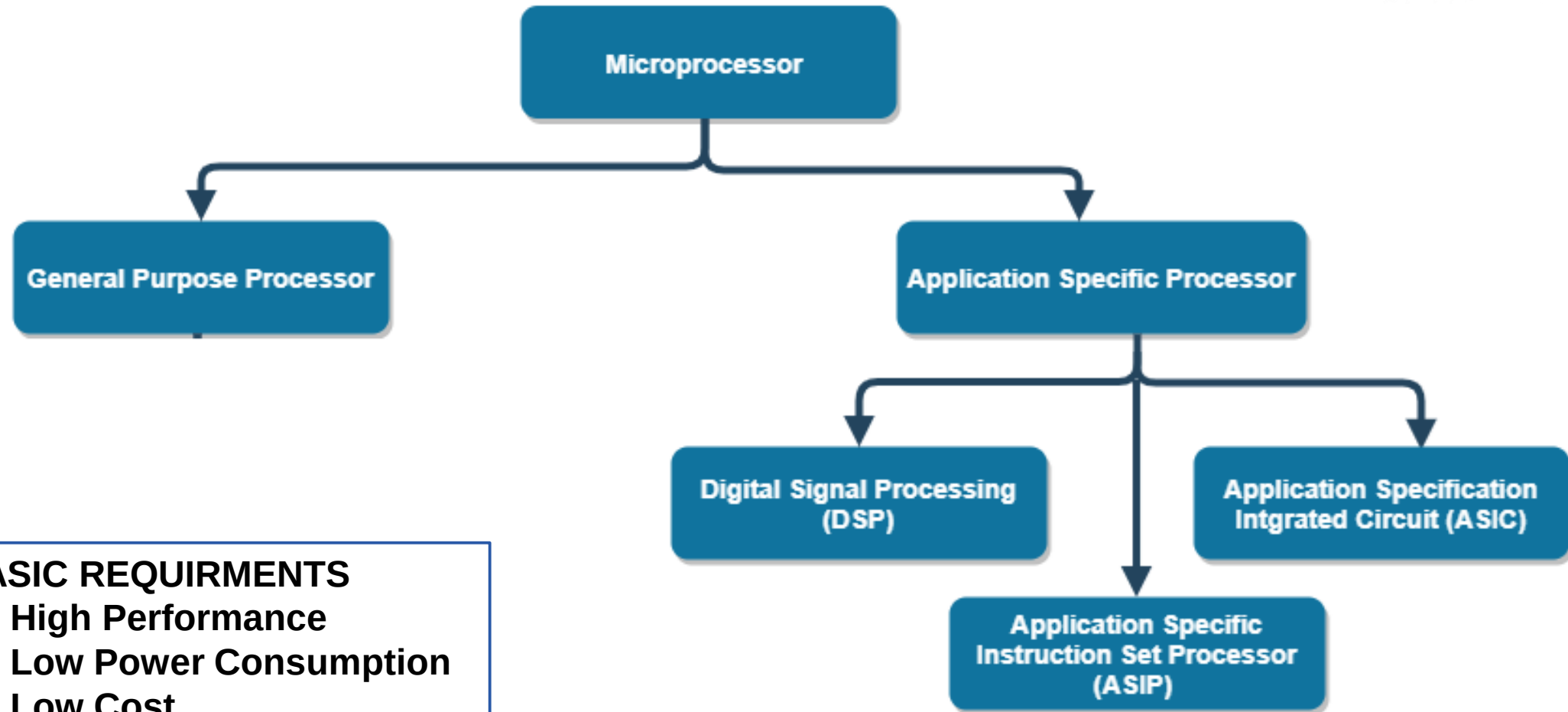


APPLICATIONS OF MICROCONTROLLER

- ❖ Automotive
- ❖ Medical electronics
- ❖ Robotics
- ❖ Consumer electronics
- ❖ Security (e-commerce, smart cards)
- ❖ Industrial automation systems

CLASSIFICATION OF MICROPROCESSORS

CLASSIFICATIONS OF MICROPROCESSORS



BASIC REQUIRMENTS

1. High Performance
2. Low Power Consumption
3. Low Cost

CLASSIFICATIONS OF MICROPROCESSORS

GENERAL PURPOSE PROCESSOR (GPP)

- ❑ Programmable device used in a variety of Applications
- ❑ GPP are designed to perform multiple tasks
- ❑ The system designer only needs to program the processor's memory to carry out the required functionality
- ❑ Biggest advantage of such system is the flexibility
- ❑ GPP lack high performance that a certain task required

CLASSIFICATIONS OF MICROPROCESSORS

GENERAL PURPOSE PROCESSOR (GPP)

- ❑ **Advantages:**
 - ❑ easy to design and use
 - ❑ design time & cost is low
 - ❑ reprogrammability
- ❑ **Disadvantages:**
 - ❑ performance is not very good
 - ❑ size is high
 - ❑ they consume much power
- ❑ **Intel “Pentium”** is the most well-known example

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION SPECIFIC PROCESSORS (ASP)

- ❑ ASPs emerged as a solution for high performance and cost-effective processors.
- ❑ This processor is designed to execute exactly one program
- ❑ Contains only the components needed to execute a single program
- ❑ An embedded designer creates a single-purpose processor by designing a custom digital circuit

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION SPECIFIC PROCESSORS (ASP)

☐ Advantages:

- ☐ performance is very good
- ☐ small size (exact to fit one solution)
- ☐ they consume little power

☐ Disadvantages:

- ☐ not very easy to design and design time is thus high
- ☐ design cost is thus higher
- ☐ reprogramming is difficult
- ☐ limited flexibility: not easy to make changes, accommodate features

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION SPECIFIC PROCESSORS (ASP) - CLASSIFICATIONS

- ❑ Digital Signal Processor (DSP): Programmable microprocessor for extensive real-time mathematical computations.
- ❑ Application Specific Instruction Set Processors (ASIP): Programmable microprocessor where hardware and instruction set are designed together for one special application.
- ❑ Application Specific Integrated Circuit (ASIC): Algorithm completely implemented in hardware.

CLASSIFICATIONS OF MICROPROCESSORS

DIGITAL SIGNAL PROCESSORS (DSP)

- ❑ DSPs are specialized microprocessors optimized for the need of performing digital signal processing.
- ❑ DSP gained their importance with the increased demand on data intensive applications such as video and internet browsing on mobile devices.
- ❑ The digital signal processors satisfy the need for powerful processor while maintaining low cost and low power consumption.

CLASSIFICATIONS OF MICROPROCESSORS

DIGITAL SIGNAL PROCESSORS (DSP)

❑ DSP Architecture Features:

- Memory architecture designed for streaming data, using DMA extensively
- Deliberate exclusion of a memory management unit (MMU)
- Bit-reversed addressing, a special addressing mode useful for calculating FFTs
- Special arithmetic operations, such as fast multiply-accumulates (MACs)
- Separate program and data memories (Harvard architecture)

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION-SPECIFIC INSTRUCTION SET PROCESSORS (ASIP)

- ❑ ASIP is typically a programmable architecture that is designed in a **specific way to perform certain tasks more efficiently**
- ❑ As the name suggests, the **Instruction set** seems to be the core characteristic of any ASIP based platform; but this is entirely not true.
- ❑ Considering a whole platform, other very important attributes like **interfaces and micro-architecture** do contribute a lot to the overall system performance.

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION-SPECIFIC INSTRUCTION SET PROCESSORS (ASIP)

- ❑ The term “Application” in ASIP is not necessarily related to **software applications**, it actually describe the class of tasks the ASIP platform was **designed to efficiently accomplish**
- ❑ This extra efficiency is not exclusively associated with **faster performance**.
- ❑ Other factors like **reduced production costs**, **simplified manufacturing process** and **less power consumption** can all be considered efficiency qualities for ASIP.

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION-SPECIFIC INTEGRATED CIRCUITS (ASIC)

- ❑ ASIC is basically an integrated circuit designed and used by a single company in a specific system.
- ❑ Example: An IC designed for a specific line of cellular phones of a company, whereby no other company can use it
- ❑ They are incredibly expensive, time-consuming, and resource-intensive to develop
- ❑ But extremely high performance coupled with low power consumption.

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION-SPECIFIC INTEGRATED CIRCUITS (ASIC)

- ❑ **Full-custom:** entirely tailor-fitted to a particular application from the very start
- ❑ **Semi-custom:** designed to allow a certain degree of modification during the manufacturing process.
- ❑ **Structured:** built from a group of 'platform slices', with a 'platform slice' being defined as a pre-manufactured device, system, or logic for that platform.
- ❑ **Gate-array:** ASIC are transistors which are predefined on the silicon wafer.

CLASSIFICATIONS OF MICROPROCESSORS

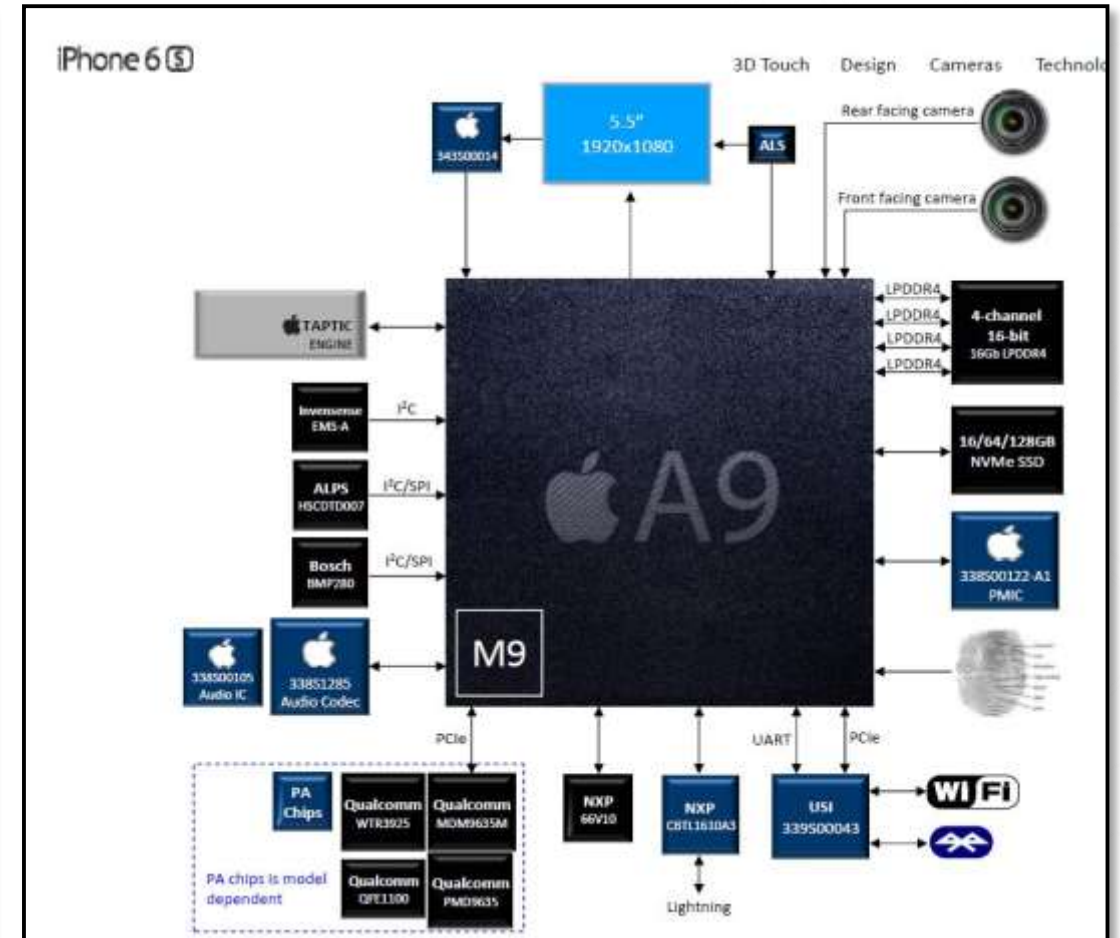
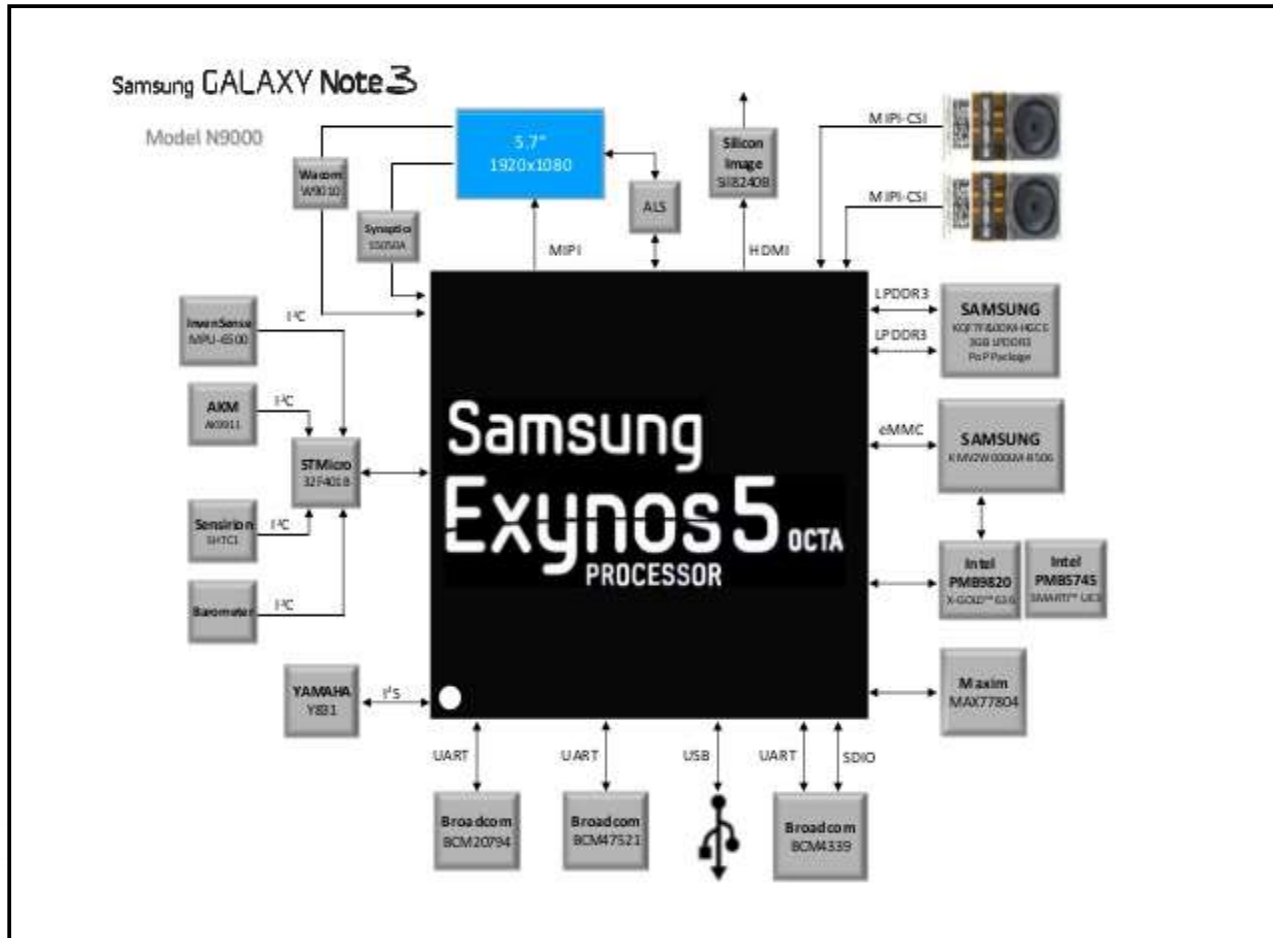
APPLICATION-SPECIFIC INTEGRATED CIRCUITS (ASIC)

❑ Applications of ASIC:

- An IC that encodes and decodes digital data using a proprietary encoding/decoding algorithm.
- A medical IC designed to monitor a specific human biometric parameter.
- An amplifier IC designed to meet certain specifications not available in standard amplifier products.
- An IC that's custom-made for a particular automated test equipment.

CLASSIFICATIONS OF MICROPROCESSORS

APPLICATION-SPECIFIC INTEGRATED CIRCUITS (ASIC) - EXAMPLE



CLASSIFICATIONS OF MICROPROCESSORS

Compares the Processors Performance vs. Their Flexibility



CLASSIFICATIONS OF MICROPROCESSORS

Different Approaches for Building Embedded System

	GPP	ASIP	ASIC
Performance	Low	High	Very High
Flexibility	Excellent	Good	Poor
HW Design	None	Large	Very Large
SW Design	Small	Large	None
Power	Large	Medium	Small
Reuse	Excellent	Good	Poor
Market	Very Large	Relatively Large	Small
Cost	Mainly on SW	SOC	Volume sensitive

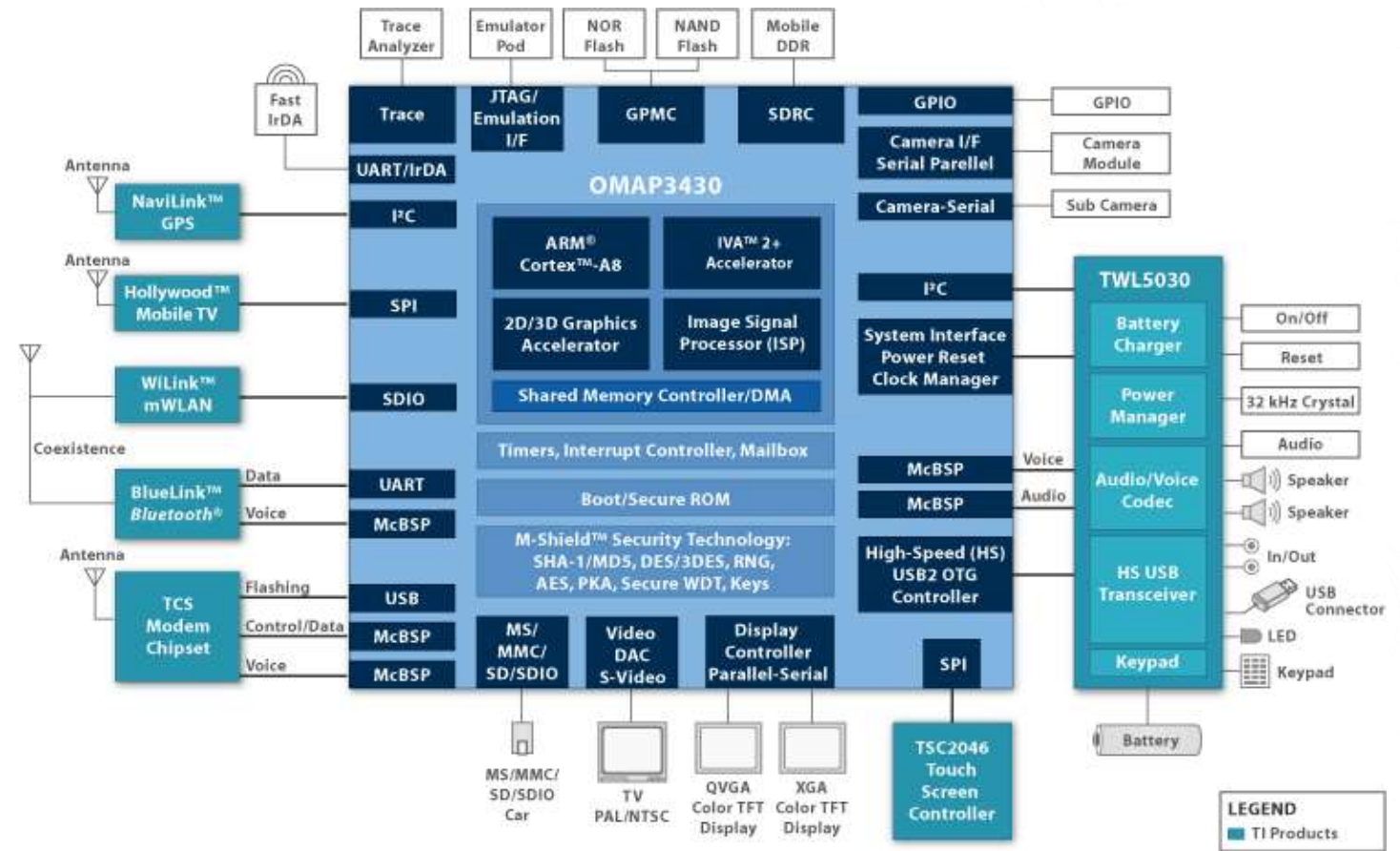
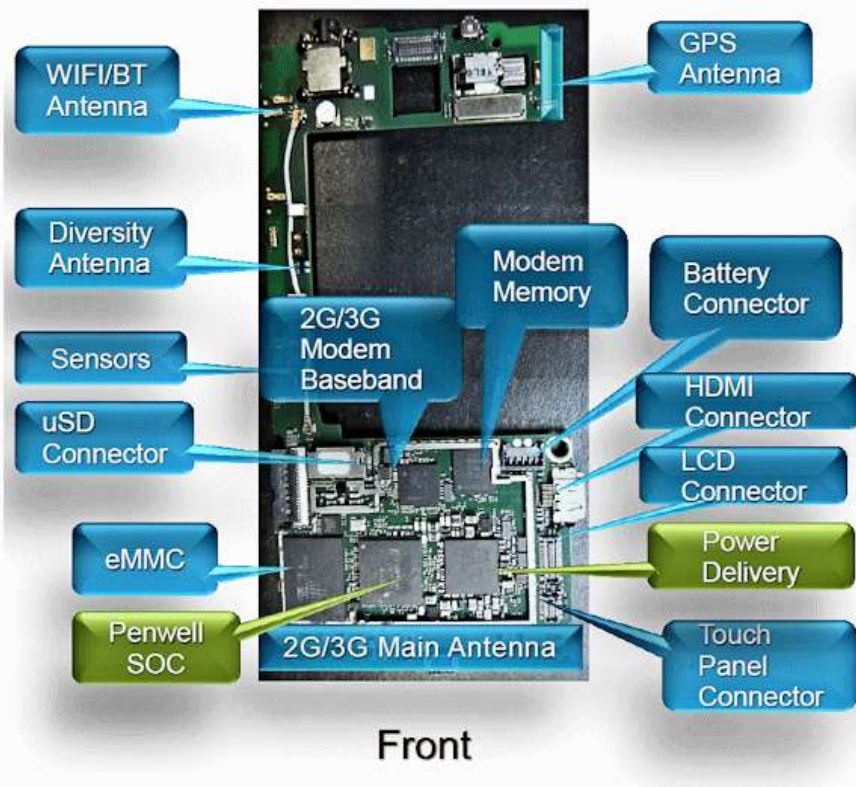
CLASSIFICATIONS OF MICROPROCESSORS

SYSTEM ON CHIP (SoC)

- ❑ SoC is an integrated circuit (IC) that takes a single platform and integrates an entire electronic system onto it. It is, exactly as its name suggests, an **entire system on a single chip**.
- ❑ SoC contains **one or more processor cores** — microprocessors (MPUs) and/or microcontrollers (MCUs) and/or digital signal processors (DSPs) — along with on-chip memory, hardware accelerator functions, peripheral functions etc.,
- ❑ SoC can perform a variety of functions including **signal processing, wireless communication, artificial intelligence etc.**

CLASSIFICATIONS OF MICROPROCESSORS

SYSTEM ON CHIP (SoC) – EXAMPLE



What's Inside a Smartphone?

CLASSIFICATIONS OF MICROPROCESSORS

SYSTEM ON CHIP (SoC) – EXAMPLE REDMI NOTE 4 (CPU-Z APP)

SOC	
	Qualcomm Snapdragon 625 2.02 GHz
Cores	8
Architecture	8x ARM Cortex-A53 @ 2.02 GHz
Revision	r0p4
Process	14 nm
Clock Speed	652 MHz – 2.02 GHz
CPU 0	2016 MHz
CPU 1	2016 MHz
CPU 2	2016 MHz
CPU 3	2016 MHz
CPU 4	2016 MHz
CPU 5	2016 MHz
CPU 6	2016 MHz
CPU 7	2016 MHz
CPU Load	57 %
GPU Vendor	Qualcomm
GPU Renderer	Adreno (TM) 506
GPU Load	0 %

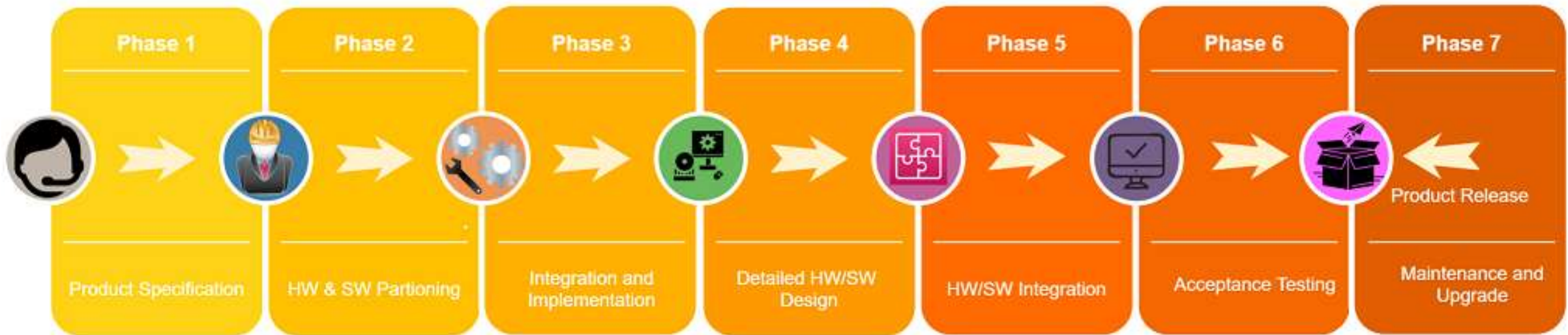
DEVICE	
Model	Xiaomi Redmi Note 4
Brand	xiaomi
Board	msm8953
Hardware	qcom
Screen Size	5.50 inches
Screen Resolution	1080 x 1920 pixels
Screen Density	400 dpi
Dimensions	151.0 x 76.0 x 8.4 mm
Weight	175 g
Total RAM	3598 MB
Available RAM	582 MB (16%)
Internal Storage	52.16 GB
Available Storage	1.37 GB (2%)
Release Date	2016-08-01

SYSTEM	
Android Version	7.0
API Level	24
Security Patch Level	2018-05-01
Bootloader	unknown
Build ID	AL1512-mido-build-20180614204110
Java VM	ART 2.1.0
OpenGL ES	3.2
Kernel Architecture	armv8l
Kernel Version	3.18.51-perf-g9a864f9 (V9.6.1.0.NCFMIFD)
Root Access	No
System Uptime	3 days, 22:56:34

SENSORS	
LSM6DS3 Accelerometer	X= 1.1 m/s² Y= 6.4 m/s² Z= 7.5 m/s²
YAS537 Magnetometer	36.0 µT
YAS537 Magnetometer Uncalibrated	153.8 µT
LSM6DS3 Gyroscope	X= 0.1 rad/s Y= 0.0 rad/s Z= 0.0 rad/s
LSM6DS3 Gyroscope Uncalibrated	X= 0.1 rad/s Y= 0.0 rad/s Z= 0.0 rad/s
LTR579 ALSPS	79.0 lux
LSM6DS3 Accelerometer -Wakeup Secondary	X= 1.0 m/s² Y= 6.4 m/s² Z= 7.5 m/s²
YAS537 Magnetometer -Wakeup Secondary	36.0 µT
YAS537 Magnetometer Uncalibrated -Wakeup Secondary	153.8 µT
LSM6DS3 Gyroscope -Wakeup Secondary	X= 0.1 rad/s Y= 0.0 rad/s Z= 0.0 rad/s
LSM6DS3 Gyroscope Uncalibrated -Wakeup Secondary	X= 0.1 rad/s Y= 0.0 rad/s Z= 0.0 rad/s
LTR579 ALSPS -Non Wakeup Secondary	

EMBEDDED SYSTEM DESIGN LIFE CYCLE

EMBEDDED SYSTEM DESIGN LIFE CYCLE



*Ref. "Embedded Systems Design: An Introduction to Processes, Tools, and Techniques"
by Arnold S. Berger*

EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-1: PRODUCT SPECIFICATION

- ☐ **Objective:** capture a formal description of the complete system
- ☐ Transforming the customer's wishes into the final product
- ☐ Customer Research Tour
- ☐ Sales and Marketing Department take the lead.
- ☐ R&D are not allowed(in general)
- ☐ Help to arrive a common Vision of the Product

EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-2: HARDWARE & SOFTWARE PARTITIONING

- ❑ **Objective:** To decide which Portion in H/W and which in S/W.
- ❑ **Example,** FPU (H/W) faster, Floating Point (S/W) slower.
- ❑ **partitioning decision is a complex optimization problem**
- ❑ **Choice of CPU impacts the partitioning decision**

EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-3: ITERATION AND IMPLEMENTATION

- ❑ **Objective:** Perform initial design before the detailed implementation
- ❑ Final stage before H/W and S/W team stop communication
- ❑ Hardware designers might be using simulation tools
- ❑ Software designers running code on single-board computers that use the target micro processor
- ❑ The Design can be extended in H/W or S/W according to convenience.

EMBEDDED SYSTEM DESIGN LIFE CYCLE

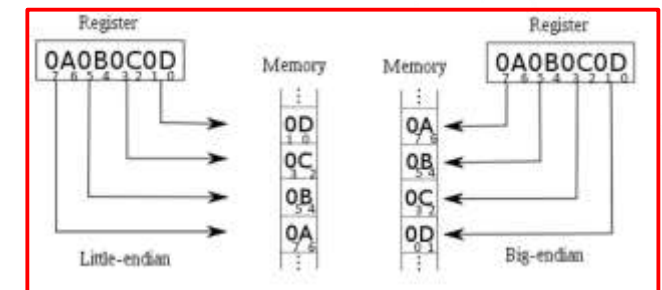
PHASE-4: DETAILED HARDWARE AND SOFTWARE DESIGN

- ❑ **Objective:** Getting the hardware and software done for the next phase
- ❑ S/W embedded developer must understand about execution environment, development tools, and run-time package.
- ❑ H/W embedded systems developer must understand about,
 - Memory usage.
 - System startup
 - Interrupts and exceptions

EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-5: HARDWARE/SOFTWARE INTEGRATION

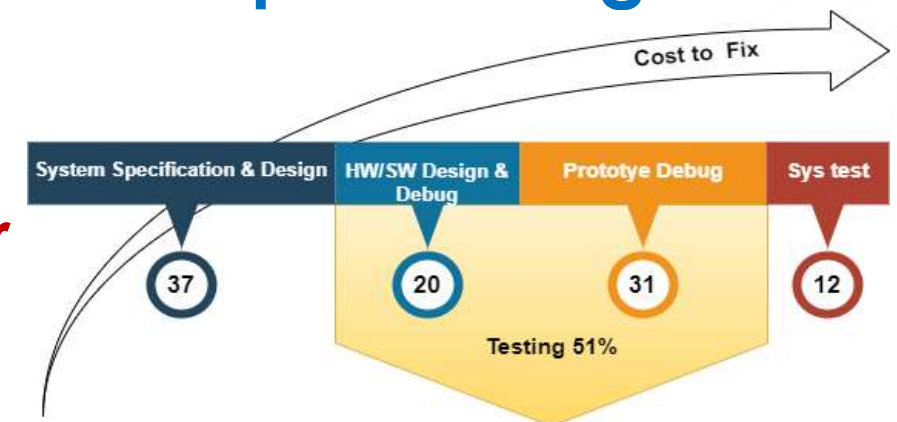
- ❑ Objective: To integrate both H/W and S/W to carryout testing
- ❑ This stage require special tools and methods to manage the complexity.
- ❑ Helps perform debugging and identify errors, Also discover whether S/W team understood the H/W description.
- ❑ Example, Big Endian/Little Endian problem



EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-6: PRODUCT TESTING AND RELEASE

- ❑ Objective : Verify designed system meet its requirement or not also ensure software doesn't crash at a critical moment
- ❑ Testing & reliability requirements-**more stringent in embedded**
- ❑ Testing must determine whether the system is performing close to its optimal capabilities
- ❑ Who does the testing? **Not the designer**



EMBEDDED SYSTEM DESIGN LIFE CYCLE

PHASE-7: MAINTAINING AND UPGRADING EXISTING PRODUCTS

- ❑ **Objective:** To maintain and add additional features on the existing product design
- ❑ **Carried out by non-members of the original design team for a particular product**
- ❑ **Experience, skills, the existing documentation, and the old product to understand the original design well enough to maintain and improve it.**

Scenario 1: Designing a Smart Home Thermostat

Question: You are tasked with developing a smart thermostat that can monitor room temperature, control HVAC systems, and be managed via a mobile app. The device must support over-the-air updates and work reliably in low-power environments.

- ◆ Which phases of the embedded system design lifecycle will be involved?
- ◆ What are the key design decisions in the hardware-software partitioning stage?

Scenario 2: Updating Firmware on a Medical Device

Question: You are leading the redesign of a wearable ECG monitor. The client requests a firmware update to add data encryption and real-time Bluetooth connectivity.

- ◆ At what stage of the lifecycle would these modifications occur?
- ◆ How would you ensure backward compatibility and system reliability?

Scenario 3: Autonomous Agricultural Robot

Question: Your team is developing an autonomous robot for crop monitoring. It must detect plant diseases using image processing and navigate fields using GPS and sensor fusion.

- ◆ How would you manage the embedded system lifecycle from concept to deployment?
- ◆ Which tasks would you assign to hardware vs. software during the architecture design?

Scenario 4: Safety-Critical Automotive Control System

Question: You are working on an embedded braking system for electric vehicles. It must respond within milliseconds and meet ISO 26262 safety standards.

- ◆ How would you incorporate safety and fault tolerance into the design lifecycle?
- ◆ Which testing strategies would be essential during integration?

Scenario 5: Industrial IoT Sensor Node

Question: An industrial client needs a network of sensor nodes to monitor machine vibrations. The system must operate for 2 years on battery, transmit minimal data, and survive harsh environments.

- ◆ What are the key considerations during the requirements and system specification stages?
- ◆ How would lifecycle design address long-term maintenance and power constraints?